

Knock Detection

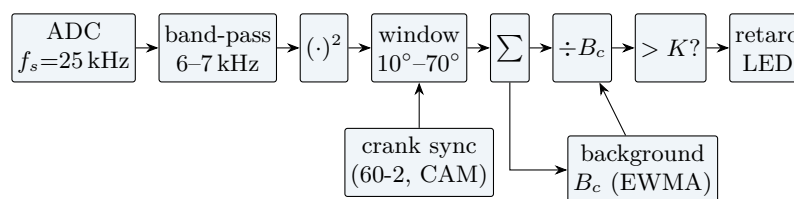
Script: Chapter 12 (capstone: angle-synchronous knock detection)

Learning goals

- Assemble the methods of the previous sheets into a complete, angle-synchronous knock detector and justify every block quantitatively.
- Evaluate a detector statistically on thousands of labelled combustions: ROC curve, area under the curve, detection rate at a fixed false-alarm rate, choice of the threshold.
- Compare energy-, FFT- and MEM-based knock features and quantify the contribution of band-pass, angle window and background normalisation.
- Run the detector in real time on the ESP32 with crank synchronisation, per-cylinder knock indication and ignition retard; optionally port it to the 8-bit Uno in fixed point.

The signal chain

The ECU does not see the engine. It sees only signals. The ECU never sees the knock – only a vibration signal that also contains noise, the combustion vibration and valve impacts. The standard knock detector of a production ECU is the chain below; every block is one of the previous sheets.



Conventions: $f_s = 25$ kHz (Sheet 10), band-pass = the 10th-order Butterworth biquad cascade of `knock_filter.h` (Sheet 10; group delay 22 samples at 6.5 kHz) or the single biquad `kf_bp2` ($B = 400$ Hz, 20 samples), window 10° – 70° after the TDC of the firing cylinder, feature $E_c = \sum_{\text{window}} y^2[n]$ for cylinder c , normalised $R_c = E_c/B_c$, knock if $R_c > K$. Ground truth: `e.knock_flag[c]` in the C model, the KNK pin of the engine-sim chip in Wokwi. Code: `code/jslinux/sheet12/` (`knockeval.c` plus `knock_filter.h`, `ar.h`, `fft.h` – use your own versions from Sheets 8, 10, 11 if you like) and `code/wokwi/sheet12_knock_esp32/`, `code/wokwi/sheet12_knock_uno/`.

Exercise 12.1 Detector design

PAPER

- Justify every block of the chain with the respective chapter: why sampling at 25 kHz (and what about anti-aliasing), why an angle window and why exactly 10° – 70° ATDC, why a band-pass, why squaring (instead of a linear operation), why summation, why a normalisation and why a threshold. Which blocks are LTI, which are not?
- Compute the window length in samples at 1500, 3000 and 6000 rpm. By how many degrees must the window be shifted to compensate the group delay of the band-pass (22 samples)? Why is a fixed sample shift correct although the window is defined in degrees?
- Energy budget. Show that a knock burst $Ae^{-t/\tau} \sin(2\pi f_1 t)$ that lies completely in the window and is passed with gain 1 contributes $E_k \approx f_s A^2 \tau / 4$ (in V^2 , sum over samples). White noise with standard deviation σ contributes on average $E_n = N_w \sigma^2 \cdot 2B_n / f_s$, where N_w is the window length and B_n the noise bandwidth of the band-pass ($B_n = \int_0^{f_s/2} |H(f)|^2 df = 1161$ Hz for the cascade). Evaluate E_k for $A = 0.2$ V and E_n for $\sigma = 0.05$ and 0.15 V at the three speeds. What do you expect for the detection performance?

- d) Without knock, E_c is approximately $E_n \cdot \chi_\nu^2 / \nu$ distributed with $\nu \approx 2B_n T_w$ degrees of freedom (T_w = window duration). Compute ν for the three speeds and, using the 99% quantiles $\chi_{\nu,0.99}^2 / \nu \approx 2.0$ ($\nu = 15.5$), 2.6 ($\nu = 7.7$), 3.4 ($\nu = 3.9$), the threshold for 1% false alarms in units of the mean noise energy. Why does a fixed threshold K give an rpm-dependent false-alarm rate?
- e) Background normalisation: propose an update rule for B_c that follows slow changes (sensor ageing, rpm, load) but is not pulled up by knocking combustions. Why one background per cylinder?
- f) Control action: on a detected knock the ignition of that cylinder is retarded by Δ_r , without knock it is advanced again by $\Delta_a \ll \Delta_r$ per combustion. Explain the asymmetry and what a false alarm costs.

Exercise 12.2 Offline evaluation with ground truth

JSLINUX

`knockeval.c` runs the virtual engine at 1500, 3000 and 6000rpm with sensor noise $\sigma = 0.05$ and 0.15 V (6 scenarios, 1000 evaluated combustions each, knock probability 0.2, knock amplitude $0.2\text{ V} \times (0.5 \dots 1.5)$, light knock) and computes for every combustion eight features, each normalised by its own per-cylinder background:

WB-seg / WB-win	wide-band energy $\sum x^2$ over the whole 180° segment / in the window
BP-seg / BP-win	band-pass (cascade) energy over the segment / in the delay-compensated window
BP2-win	as BP-win with the single biquad ($B = 400\text{ Hz}$)
FFT / FFT-rect	power 6–7 kHz of the zero-padded FFT of the raw window samples, Hann / no window
MEM	power 6–7 kHz of the Burg AR(8) spectrum of the raw window samples

```
gcc -O2 -o knockeval knockeval.c -lm && ./knockeval [knock_amp] [ncomb]
```

- a) Complete the window logic: integrate x^2 in the window 10°–70° ATDC and y^2 (cascade, single biquad) while the *delayed* angle is in the window.
- b) Complete the background update (clipped EWMA, faster during the first 16 combustions of each cylinder).
- c) Complete the ROC computation in `roc()`: the combustions are sorted by decreasing feature value; every threshold between two values gives a point (FPR, TPR). Compute the AUC (trapezoidal rule) and the detection rate at $\leq 1\%$ false alarms.
- d) Complete the FFT and MEM features (`fft.h`, `ar.h`).
- e) Run the program and interpret the AUC and detection tables: what is the contribution of the band-pass, of the angle window, of the delay compensation, of the normalisation (compare the pooled ROC with and without)? Plot `roc.csv`.
- f) Choose a threshold K for 1% false alarms over all scenarios and compare the per-scenario thresholds with your prediction from 12.1 d).
- g) Compare the energy, FFT and MEM features. Why is the Hann-windowed FFT so much worse at 1500rpm than the rectangular one? Which feature would you implement on the microcontroller, and why?

Exercise 12.3 Real-time knock detector

WOKWI

Open [code/wokwi/sheet12_knock_esp32/](https://code.wokwi.com/projects/sheet12_knock_esp32/) (ESP32 + engine-sim chip, 4 knock LEDs, retard LED, logic analyzer). Pins: KNOCK → GPIO34, CRANK → 18, CAM → 19, TDC → 21 (only to check the synchronisation), KNK → 22 (ground truth, only for the evaluation); LEDs cylinder 1–4 on 25,

26, 27, 14; retard PWM on 13. The sketch samples at 25 kHz (deadline loop), filters with `kf_sos` and prints one line `cyl,rpm,R,knock,truth,retard` per combustion and every 200 combustions the confusion counts.

- Crank sync.* Complete the 60-2 decoder in the CRANK interrupt (gap detection: a tooth period more than twice the previous one; revolution from the CAM level at tooth 10) and the angle interpolation `crank_angle()`. The sketch compares the decoded angle at every TDC pulse with 120/300/480/660° and prints the mean error. What limits the accuracy?
- Window.* Complete the integration of y^2 while the delayed angle (ring buffer `th_hist`, `KF_DELAY = 22` samples) is in the window.
- Decision and action.* Complete `finish_combustion()`: $R = E/B_c$, knock if $R > K$ ($K = 2$ from 12.2), background update, ignition retard (+1.5° per knock, -0.05° per knock-free combustion, max. 9°).
- Run the detector at 1500, 3000, 6000 rpm with knock amplitude 1.0 and 0.2 V and noise 50 and 150 mV (knock probability 20 %). Report detection and false-alarm rates and compare them with 12.2. Watch the LEDs and the retard output.
- Estimate the processing time per sample on the ESP32 and the UART load at 6000 rpm. Why must the sketch not print the samples?
- The engine-sim chip has no spark input: the retard does not influence the knock probability. Describe how a closed knock control loop would behave and how you would test it.

Exercise 12.4 Port to the Arduino Uno

CHALLENGE

Port the detector to the Uno (`code/wokwi/sheet12_knock_uno/`, starting from the Q15 biquad of Sheet 10): Timer1 ISR at 25 kHz, single biquad `kf_bp2_q14`, window from the TDC pulse (period in samples of the previous combustion, cylinder 1 = TDC while CAM is HIGH), energy $\sum(y^2 \gg 8)$ in 32 bit, integer background $B += (\min(E, 3B) - B) \gg 4$, decision $E > 2B$ without division. Report detection and false-alarm rates as in 12.3 and the limits of the port.

Deliverables

12.1 on paper (block justification, numbers for b)–d)). `knockeval`: AUC and detection tables, pooled ROC plot (BP-win, WB-win, FFT, MEM), chosen threshold with justification. Wokwi: the confusion counts for the settings of 12.3 d) and a screenshot of the LEDs; a short discussion (max. one page) of the remaining weaknesses of your detector.